

Architecting Solutions on AWS

Module 1

Designing a serverless web backend on AWS

Course Introduction
Video • 3 min

Welcome to the Course
Reading • 10 min

Course Feedback
Video • 1 min

Pre-Course Survey
Reading • 1 min

Pre-Course Survey
Ungraded Plugin • 15 min

Week 1 Introduction
Video • 3 min

Introduction
Reading • 25 min

Customer #1: Use Case and Requirements
Video • 8 min

Customer #1: Requirements Breakdown
Video • 3 min

Selecting a Serverless Compute Service
Video • 9 min

AWS Lambda Exploration
Video • 7 min

Compute on AWS
Reading • 25 min

Choosing an AWS Database Service
Video • 10 min

DynamoDB Exploration
Video • 6 min

Databases on AWS
Reading • 25 min

Building Event-Driven Architectures
Video • 9 min

SNS Exploration
Video • 3 min

Event-Driven Architectures on AWS
Reading • 25 min

Decoupling AWS Solutions
Video • 7 min

SQS Exploration
Video • 5 min

Decoupling Solutions on AWS
Reading • 25 min

Customer-Centric Serverless Architecture Design
Role Play • 15 min

Customer #1: Solution Overview
Video • 4 min

Week Wrap-up: Taking this Architecture to the Next Level
Video • 6 min

Architecture Optimizations for Week 1
Reading • 25 min

Exercise 1 Walkthrough
Video • 24 min

Challenge: Building a Proof of Concept
Reading • 30 min

Week 1 Assessment
Graded Assignment • 25 min

Mid-Course Survey
Reading • 1 min

Mid-Course Survey
Ungraded Plugin • 15 min

Module 2

Designing a serverless data analytics solution on AWS

Module 3

Designing a hybrid solution for container based workloads on AWS

Module 4

Designing a solution following account governance and management best... practices

Compute on AWS

When you consider what compute service to use for a specific use case, you should ensure that you are up to date on any new AWS service or feature releases. To review a high-level overview of the different AWS compute services AWS, see [Compute on AWS - Compute for any workload](#).

AWS Lambda

For this weeks architecture, we have chosen AWS Lambda as the compute service due to it's serverless nature and ability to support a web backend. Lambda is a compute service that provides serverless compute functions that run in response to events or triggers. When an event or trigger is detected, a Lambda function is spun up in its own secure and isolated runtime environment, which is called an execution environment. Lambda functions can run for up to 15 minutes. Any processes that need longer than 15 minutes to run should use other compute services on AWS for hosting. Each execution environment stays active for a period of time, and then it shuts down on its own.

When you use Lambda, you are responsible only for your code, which can make it easier to optimize for operational efficiency and low operational overhead. Lambda manages the compute fleet, which offers a balance of memory, CPU, network, and other resources to run your code. Because Lambda manages these resources, you can't log in to compute instances or customize the operating system on the provided runtimes. Lambda performs operational and administrative activities on your behalf, including managing capacity, monitoring, and logging your Lambda functions.If you need to manage your own compute resources, AWS has other compute services that can meet your needs. For example:

- Amazon Elastic Compute Cloud (Amazon EC2) offers a wide range of EC2 instance types to choose from. With Amazon EC2, you can customize operating systems, settings for network and security, and the entire software stack. You are responsible for provisioning capacity, monitoring fleet health and performance, and using Availability Zones for fault tolerance.
- AWS Elastic Beanstalk is a service that you can use to deploy and scale applications on Amazon EC2. You retain ownership and full control over the underlying EC2 instances.

Lambda can be used for virtually any application or backend that requires compute and that runs in under 15 minutes. Common use cases are web backends, Internet of Things (IoT) backends, mobile backends, file or data processing, stream or message processing, and more. Lambda is a good choice for use cases where the requirements include reducing operational overhead, optimizing for cost, or optimizing for performance efficiency. Lambda works well for these use cases because it's a managed service and you only pay for what you use. There are no idling resources when working with AWS Lambda, which means that each Lambda function is highly performant and cost efficient.

- To gain a deeper understanding of Lambda, see [AWS Lambda FAQs](#).
- To learn more about Lambda, see the [AWS Lambda Developer Guide](#) or the [AWS Lambda Operator Guide](#).
- To learn more about architecting and best practices for Lambda, see [Architecture and Best Practices](#).
- For a list of technical talks that cover Lambda, see [AWS Lambda - Technical Talks](#).
- For a list of technical tutorials that use Lambda, see [AWS Lambda - Workshops & Tutorials](#).

Amazon API Gateway

After Morgan selected Lambda for the compute backend, she needed to find a way to expose the backend Lambda function. Amazon API Gateway integrates with Lambda, thus providing a way to expose the backend service without exposing to the open internet. This week's customer might decide to add authentication to API Gateway to secure it further.

API Gateway is a fully managed service that makes it easier for developers to create, publish, maintain, monitor, and secure APIs at any scale. APIs act as the front door for applications, so that the applications can access data, business logic, or functionality from your backend services. By using API Gateway, you can create RESTful APIs and WebSocket APIs that enable real-time two-way communication applications. API Gateway supports containerized and serverless workloads, as well as web applications.

API Gateway handles all the tasks involved in accepting and processing up to hundreds of thousands of concurrent API calls, including traffic management, CORS support, authorization and access control, throttling, monitoring, and API version management. API Gateway has no minimum fees or startup costs. You pay for the API calls you receive and the amount of data transferred out and, with the API Gateway tiered pricing model, you can reduce your cost as your API usage scales.

- For more information about Amazon API Gateway, see [Amazon API Gateway](#) or [Amazon API Gateway FAQs](#).

Amazon EC2

Amazon EC2 is a service that provides resizable compute capacity in the cloud, which means that it provides virtual machines in the cloud. Amazon EC2 is a flexible service that offers multiple instance types, sizes, and pricing models to meet specific requirements. Because you can choose your operating system and configurations for your instance, you can configure Amazon EC2 to work with virtually any workload.

You can use Amazon EC2 when you want to run applications on AWS, but still want to retain control over the underlying infrastructure. Morgan didn't choose Amazon EC2 as the compute service for this customer's use case because of the operational overhead that Amazon EC2 requires. You have a lot of control over Amazon EC2, but that control also means that you will have overhead for managing the service. The customer had a straightforward use case and was willing to rewrite the code to use Lambda. The customer also had a spiky demand for their workload. Thus, choosing a service such as Lambda minimizes idling resources during low volume times, which can be more difficult to achieve with Amazon EC2.

- For a beginner tutorial about Amazon EC2, see [Get started with Amazon EC2 Linux instances](#).
- To gain a deeper understanding of Amazon EC2, see [What is Amazon EC2?](#) or [Amazon EC2 FAQs](#).

AWS container services

Container management tools can be divided into three categories: registry, orchestration, and compute. AWS offers services that give you a secure place to store and manage your container images, orchestration that manages when and where your containers run, and flexible compute engines to power your containers. AWS can help manage your containers and their deployments for you, so you don't have to worry about the underlying infrastructure. No matter what you're building, AWS makes it easy and efficient to build with containers.

Container services were not chosen for this architecture because the customer did not want to integrate this technology into their stack. So, even though running a container on Amazon ECS using AWS Fargate as the compute platform would technically work it was not chosen because of other customer preferences.

Amazon ECS

Amazon Elastic Container Service (Amazon ECS) is a fully managed container orchestration service that you can use to deploy, manage, and scale containerized applications. It integrates with the rest of the AWS Cloud to provide a secure and easy-to-use solution for running container workloads in the cloud or on premises. Key features of Amazon ECS:

- Serverless by default with AWS Fargate: Fargate is built into Amazon ECS, and it reduces the time you need to spend on managing servers, handling capacity planning, or figuring out how to isolate container workloads for security. With Fargate, you define your application's requirements and select Fargate as your launch type in the console or AWS Command Line Interface (AWS CLI). Then, Fargate takes care of all the scaling and infrastructure management that's needed to run your containers.
- Security and isolation by design: Amazon ECS natively integrates with the tools you already trust for security, identity, and management and governance. This can help you get to production quickly and successfully. You can assign granular permissions for each of your containers, giving you a high level of isolation when you build your applications. You can launch your containers with the security and compliance levels that you have come to expect from AWS.
- Autonomous control plane operations: Amazon ECS is a fully-managed container orchestration service, with AWS configuration and operational best practices built-in—with no control plane, nodes, or add-ons for you to manage. It natively integrates with both AWS and third-party tools to make it easier for teams to focus on building the applications, not the environment.

Amazon EKS

Amazon Elastic Kubernetes Service (Amazon EKS) is a managed service that you can use to run Kubernetes on AWS without needing to install, operate, and maintain your own Kubernetes control plane or nodes. Kubernetes is an open-source system for automating the deployment, scaling, and management of containerized applications. Amazon EKS offers the following features:

- It runs and scales the Kubernetes control plane across multiple AWS Availability Zones to ensure high availability.
- It also automatically scales control plane instances based on load, detects and replaces unhealthy control plane instances, and provides automated version updates and patching for them.
- It is integrated with many AWS services to provide scalability and security for your applications, including the following capabilities:
 - Amazon Elastic Container Registry (Amazon ECR for container images).
 - Elastic Load Balancing for load distribution.
 - AWS Identity and Access Management (IAM) for authentication.
 - Amazon Virtual Private Cloud (VPC) for isolation.
- It runs up-to-date versions of Kubernetes, so you can use all of the existing plugins and tooling from the Kubernetes community.

Applications that run on Amazon EKS are fully compatible with applications that run on any standard Kubernetes environment—it doesn't matter whether they run in on-premises data centers or public clouds. This means that you can migrate any standard Kubernetes application to Amazon EKS with virtually no code modification.

AWS Fargate

AWS Fargate is a technology that you can use with Amazon ECS to run containers without managing servers or clusters of EC2 instances. AWS Fargate reduces your need to provision, configure, or scale clusters of virtual machines to run containers. Thus, it also minimizes your need to choose server types, decide when to scale your clusters, or optimize cluster packing.

When you run your tasks and services with the Fargate launch type, you package your application in containers, specify the CPU and memory requirements, define networking and IAM policies, and launch the application. Each Fargate task has its own isolation boundary and doesn't share the underlying kernel, CPU resources, memory resources, or elastic network interface with another task.

With Amazon ECS on AWS Fargate capacity providers, you can use both Fargate and Fargate Spot capacity with your Amazon ECS tasks. With Fargate Spot, you can run interruption-tolerant Amazon ECS tasks at a discounted rate, compared to the Fargate price. Fargate Spot runs tasks on spare compute capacity. When AWS needs the capacity back, your tasks will be interrupted with a 2-minute warning notice.

- To learn more about Amazon EKS, see the [Amazon EKS User Guide](#).
- To learn more about AWS container services, see [Containers on AWS](#).
- To learn more about AWS Fargate, see [Amazon ECS on AWS Fargate](#).
- For hands-on tutorials using Amazon ECS, see [Amazon ECS Workshop](#).

Mark as completed

Like Dislike Report an issue

coach



Hi, Venkatesh!
How can I help?

Explain this topic in simple terms

Give me a summary

Give me practice questions

Give me real-life examples

Ask me anything



Go to next item